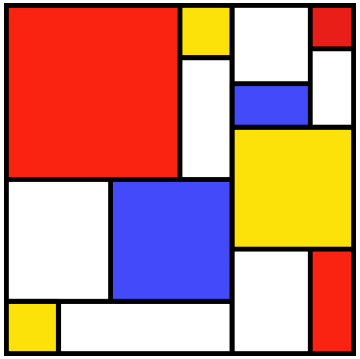
SLiM

Workshop Series



The Chromosome Hierarchy

Benjamin C. Haller

Messer Lab, Cornell University

The `initialize()` callback

```
// set up a simple neutral simulation
initialize()
{
   // set the overall mutation rate
   initializeMutationRate(1e-7);

   // m1 mutation type: neutral
   initializeMutationType("m1", 0.5, "f", 0.0);

   // g1 genomic element type: uses m1 for all mutations
   initializeGenomicElementType("g1", m1, 1.0);

   // uniform chromosome of length 100 kb
   initializeGenomicElement(g1, 0, 99999);

   // uniform recombination along the chromosome
   initializeRecombinationRate(1e-8);
}
```

Callbacks

```
initialize()
{
   ...
}
```

- **Callbacks** are script blocks called by SLiM

Callbacks

```
initialize()
{
   ...
}
```

- **Callbacks** are script blocks called by SLiM
- Various types:
  - `initialize()` callbacks are called at the **start**
  - `early()` events are called **early** each tick
  - `late()` events are called **late** each tick
  - `mutationEffect()`, `mateChoice()`, `modifyChild()`, `recombination()`, `reproduction()`, `...`

The `initialize()` callback

```
// set up a simple neutral simulation
initialize()
{
   // set the overall mutation rate
   initializeMutationRate(1e-7);

   // m1 mutation type: neutral
   initializeMutationType("m1", 0.5, "f", 0.0);

   // g1 genomic element type: uses m1 for all mutations
   initializeGenomicElementType("g1", m1, 1.0);

   // uniform chromosome of length 100 kb
   initializeGenomicElement(g1, 0, 99999);

   // uniform recombination along the chromosome
   initializeRecombinationRate(1e-8);
}
```

Function calls

```
// set the overall mutation rate
initializeMutationRate(1e-7);
```

- Function calls *do something* (perform an action)
- Functions are passed *parameters*
- Here, the mutation rate is initialized to `1e-7`

Function calls

```
// set the overall mutation rate
initializeMutationRate(1e-7);
```

- Function calls *do something* (perform an action)
- Functions are passed *parameters*
- Here, the mutation rate is initialized to `1e-7`
- Several types of functions:
  - Eidos provides more than 180 built-in functions
  - SLiM provides a few built-in functions also
  - user-defined functions can be written in script

Mutation types

```
// m1 mutation type: neutral
initializeMutationType("m1", 0.5, "f", 0.0);
```

- Mutation types
  - a **category of mutation**: neutral, beneficial, etc.
  - defined by a *distribution of fitness effects* (DFE)
  - the DFE provides selection coefficients (s)
  - also have a dominance coefficient (h)
  - here, a fixed DFE (`"f"`) with s=0.0, h=0.5
  - fitness effects are $1+s$ (AA) and $1+hs$ (Aa)
  - named; this mutation type will be `m1`

Genomic element types

```
// g1 genomic element type: uses m1 for all mutations
initializeGenomicElementType("g1", m1, 1.0);
```

- Genomic element types
  - represent a **category of genomic region**
  - exon, intron, non-coding region, UTR, etc.
  - draw mutations from a set of mutation types
  - here, drawing just from `m1` (weight of `1.0`)
  - named; this genomic element type will be `g1`

Genomic elements

```
// uniform chromosome of length 100 kb
initializeGenomicElement(g1, 0, 99999);
```

- Genomic elements
  - represent a **specific region of a chromosome**
  - span some interval [start, end], in base positions
  - draw mutations from one genomic element type
  - this genomic element is of type `g1`
  - this genomic element spans positions [`0`, `99999`]

The chromosome hierarchy

- A **chromosome** object defines a genetic structure
- It is a sequence of **genomic elements**

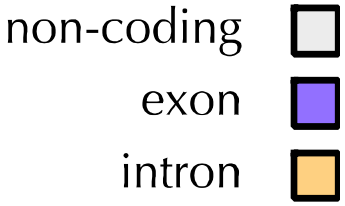
Chromosome: a mosaic of genomic elements

The chromosome hierarchy

- A **chromosome** object defines a genetic structure
- It is a sequence of **genomic elements**
- Each element belongs to a **genomic element type**

Chromosome: a mosaic of genomic elements

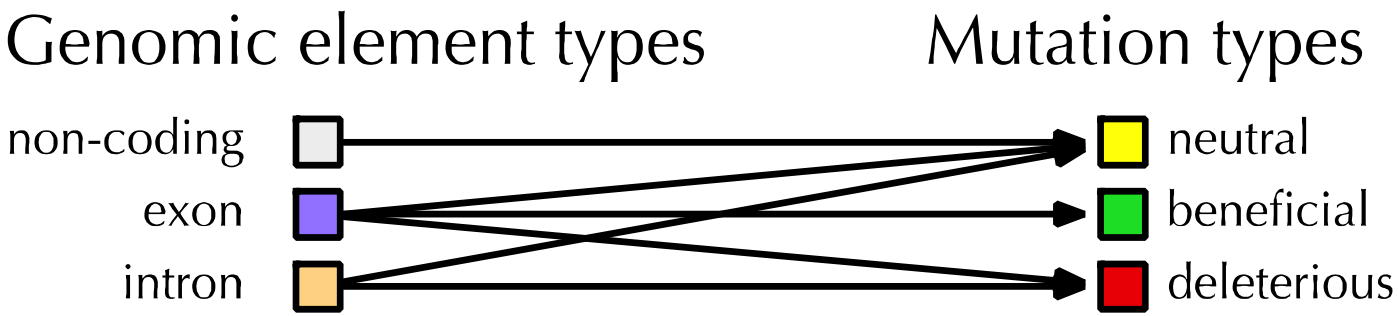
Genomic element types



The chromosome hierarchy

- A **chromosome** object defines a genetic structure
- It is a sequence of **genomic elements**
- Each element belongs to a **genomic element type**
- Each type draws from a DFE (**mutation type**)

Chromosome: a mosaic of genomic elements



The `initialize()` callback

```
// set up a simple neutral simulation
initialize()
{
   // set the overall mutation rate
   initializeMutationRate(1e-7);

   // m1 mutation type: neutral
   initializeMutationType("m1", 0.5, "f", 0.0);

   // g1 genomic element type: uses m1 for all mutations
   initializeGenomicElementType("g1", m1, 1.0);

   // uniform chromosome of length 100 kb
   initializeGenomicElement(g1, 0, 99999);

   // uniform recombination along the chromosome
   initializeRecombinationRate(1e-8);
}
```

Callbacks

- **Callbacks** are script blocks called by SLiM
- Various types:
  - `initialize()` callbacks are called at the **start**
  - `early()` events are called **early** each tick
  - `late()` events are called **late** each tick
  - **...**

Callbacks

- **Callbacks** are script blocks called by SLiM
- Various types:
  - `initialize()` callbacks are called at the **start**
  - `early()` events are called **early** each tick
  - `late()` events are called **late** each tick
  - **...**
- We defined the *model structure* in `initialize()`
- Now we'll define some *ecology* with `early()`

The `early()` event

```
// create a population of 500 individuals
1 early()
{
    sim.addSubpop("p1", 500);
}
```

- Declares an `early()` event (callback)
- Declared to run in tick `1`
- Makes a *method call*: `target.method()`
  - like a function call, but sent to a *target object*
  - this target object, `sim`, represents the simulation
  - defines a new subpopulation, `p1`, of size `500`

The `late()` event

```
// run to tick 10000
10000 late()
{
    sim.outputFixedMutations();
    sim.simulationFinished();
}
```

- Declares a `late()` event (callback)
- Declared to run in tick `10000`
- Makes a method call to output substitutions
  - (no parameters here, but you use `()` anyway)
- Makes a method call to stop the simulation
  - (the simulation would stop anyway)

The complete model

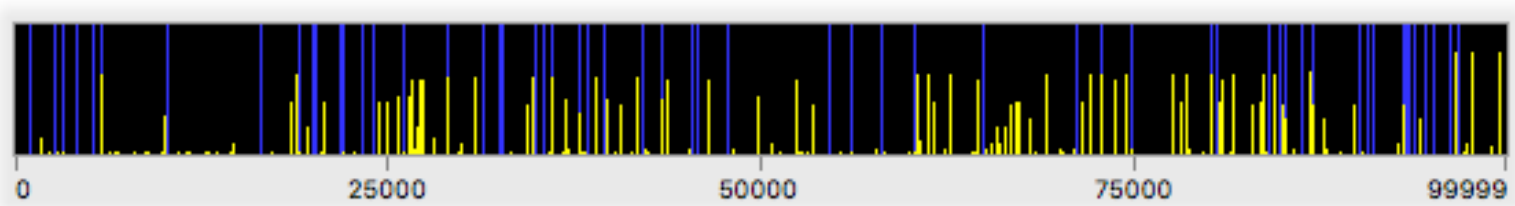
```
// set up a simple neutral simulation
initialize()
{
	// set the overall mutation rate
	initializeMutationRate(1e-7);

	// m1 mutation type: neutral
	initializeMutationType("m1", 0.5, "f", 0.0);

	// g1 genomic element type: uses m1 for all mutations
	initializeGenomicElementType("g1", m1, 1.0);

	// uniform chromosome of length 100 kb
	initializeGenomicElement(g1, 0, 99999);

	// uniform recombination along the chromosome
	initializeRecombinationRate(1e-8);
}

// create a population of 500 individuals
1 early()
{
	sim.addSubpop("p1", 500);
}

// run to tick 10000
10000 late()
{
	sim.outputFixedMutations();
	sim.simulationFinished();
}
```

Model output

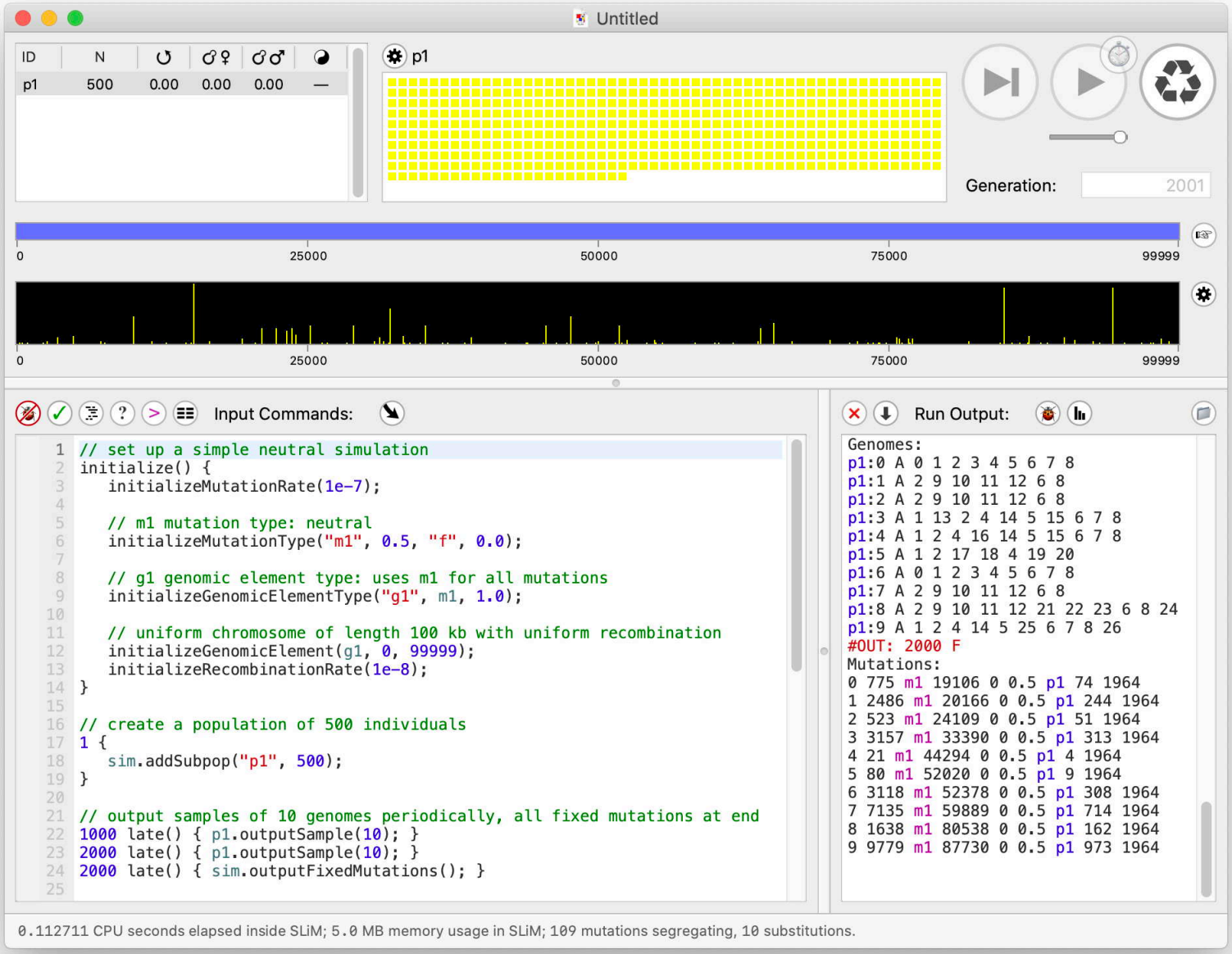
- From `outputFixedMutations()`:

```
#OUT: 10000 10000 F
Mutations:
0 119 m1 80553 0 0.5 p1 14 1219
1 2404 m1 39498 0 0.5 p1 246 1450
...
56 70547 m1 91029 0 0.5 p1 7026 9424
57 69695 m1 95168 0 0.5 p1 6945 9424
```

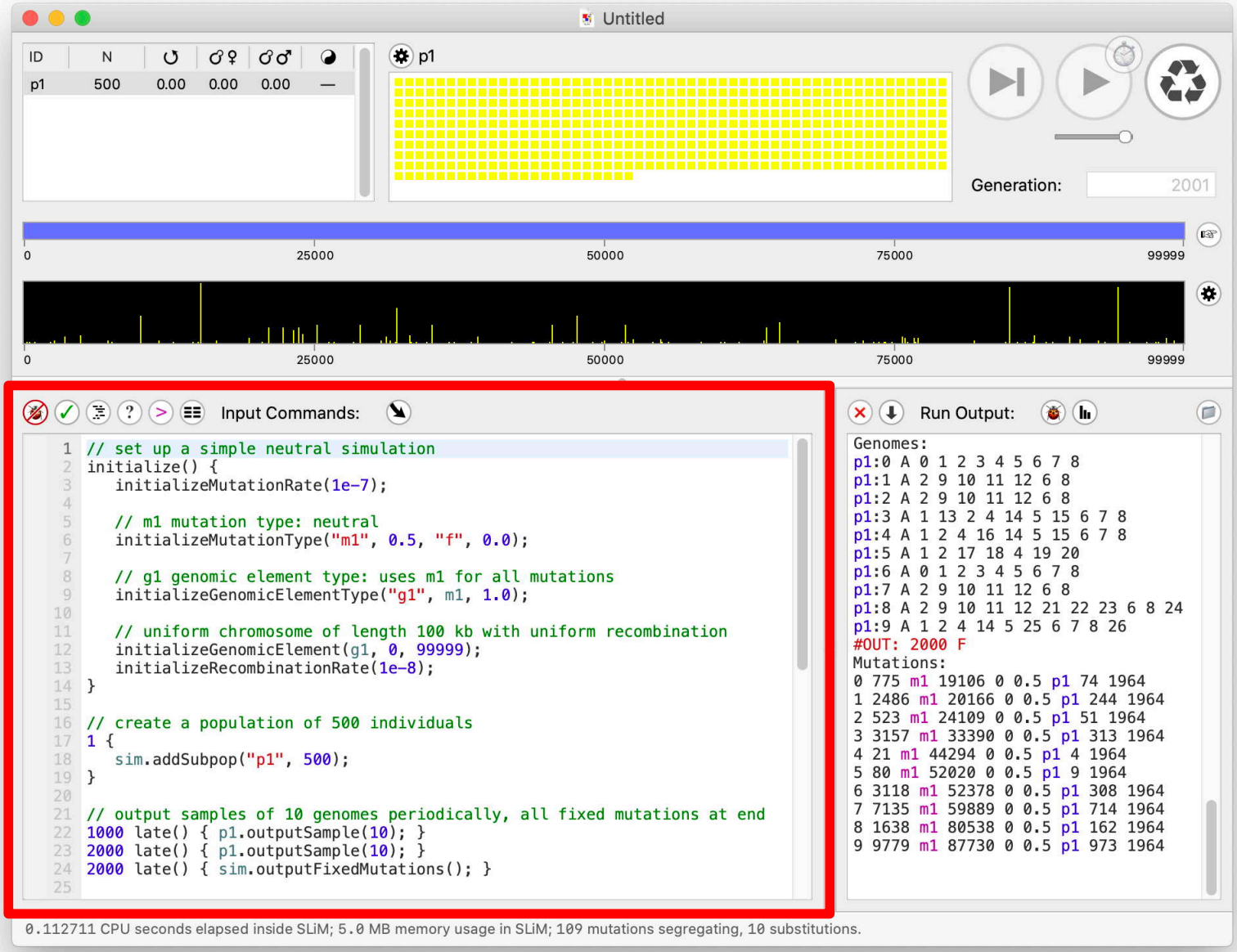
- In SLiMgui:



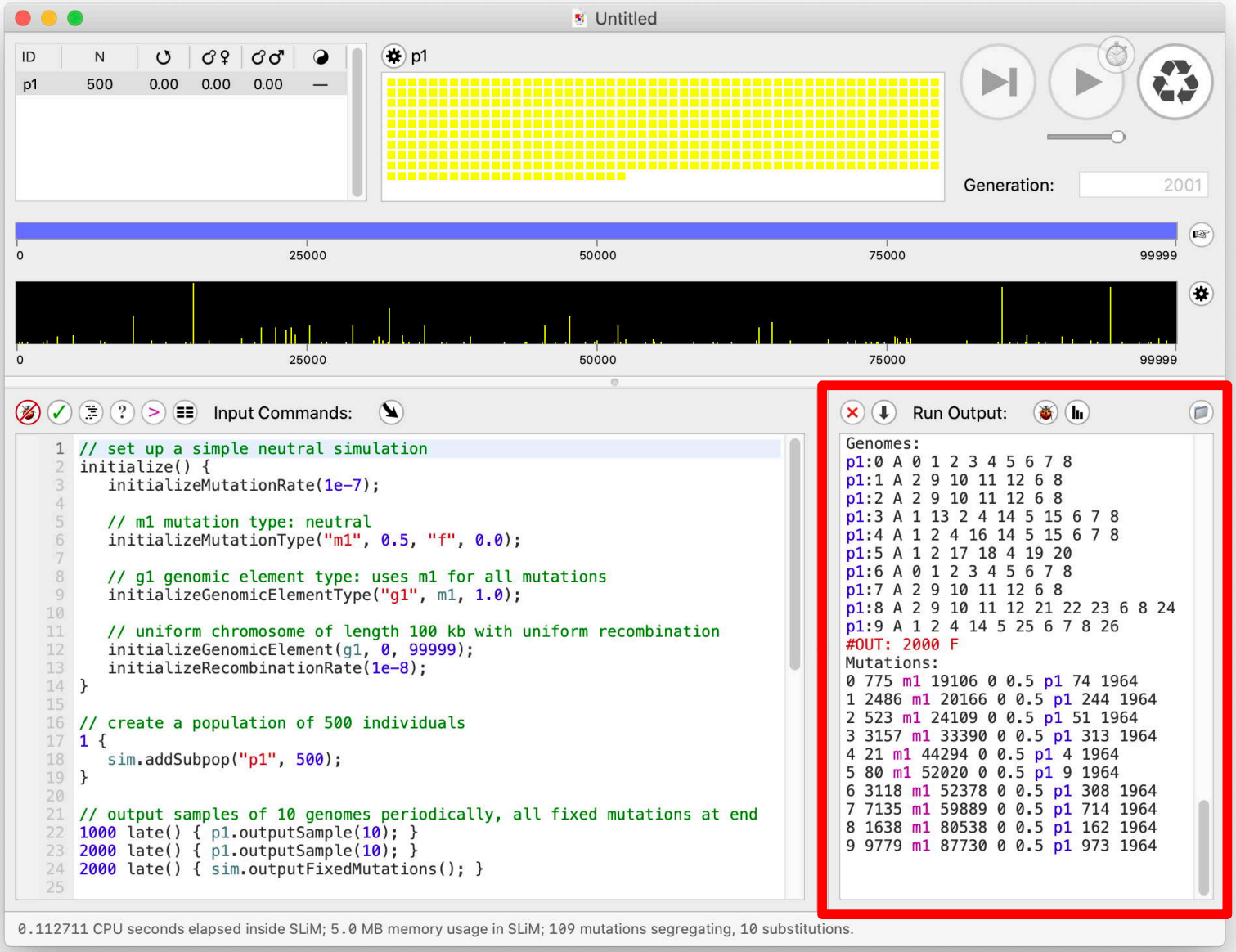
Modeling in SLiMgui



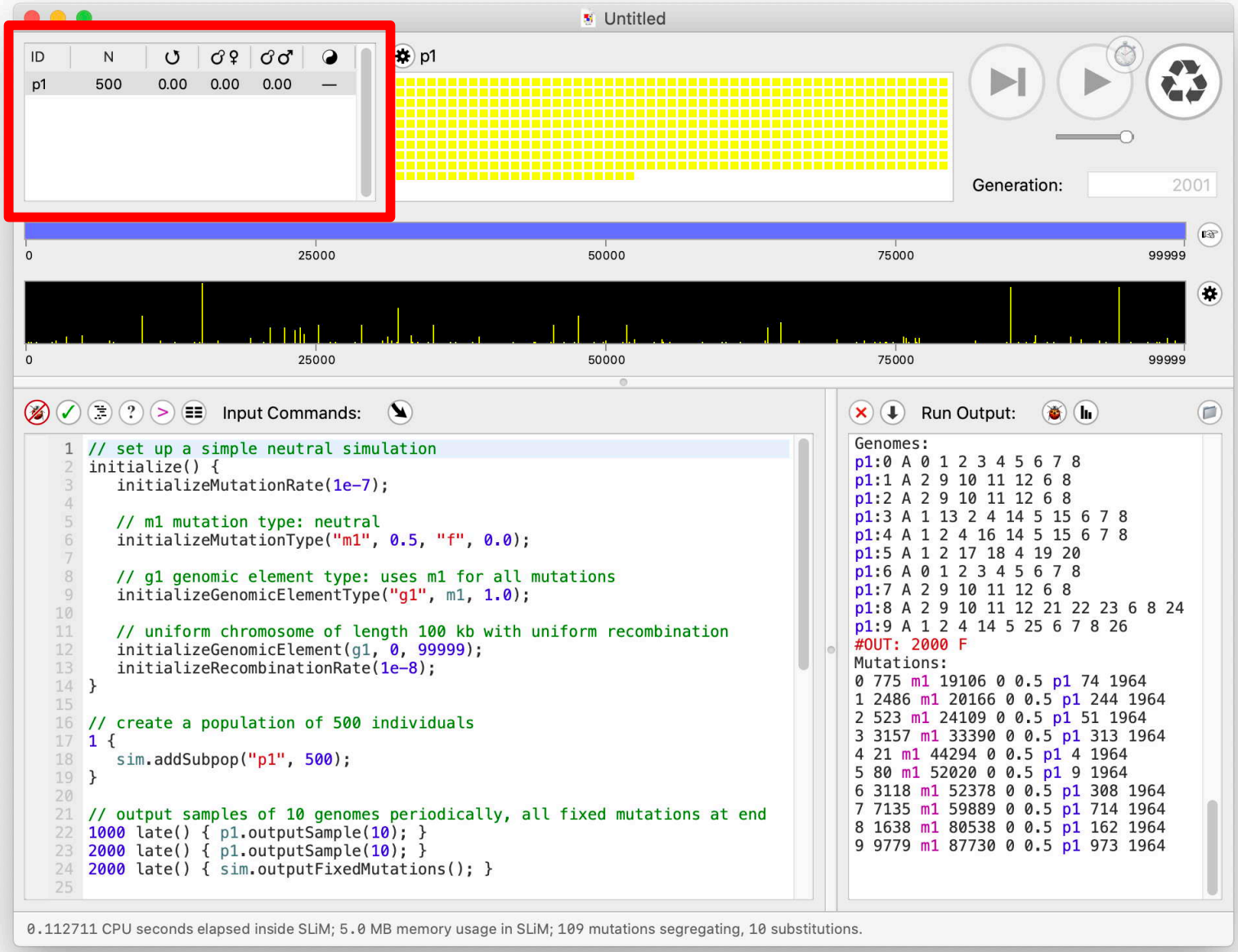
Modeling in SLiMgui



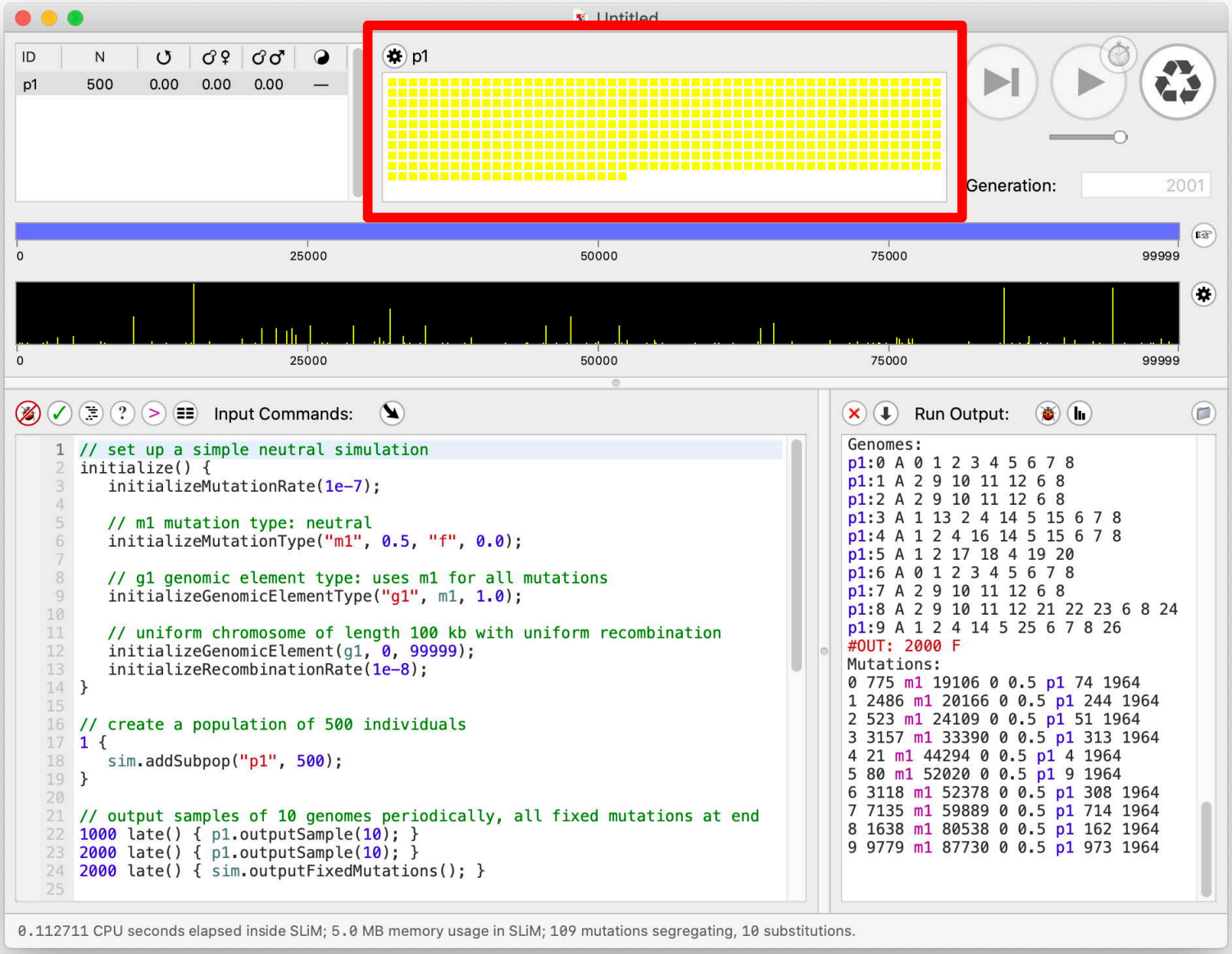
Modeling in SLiMgui



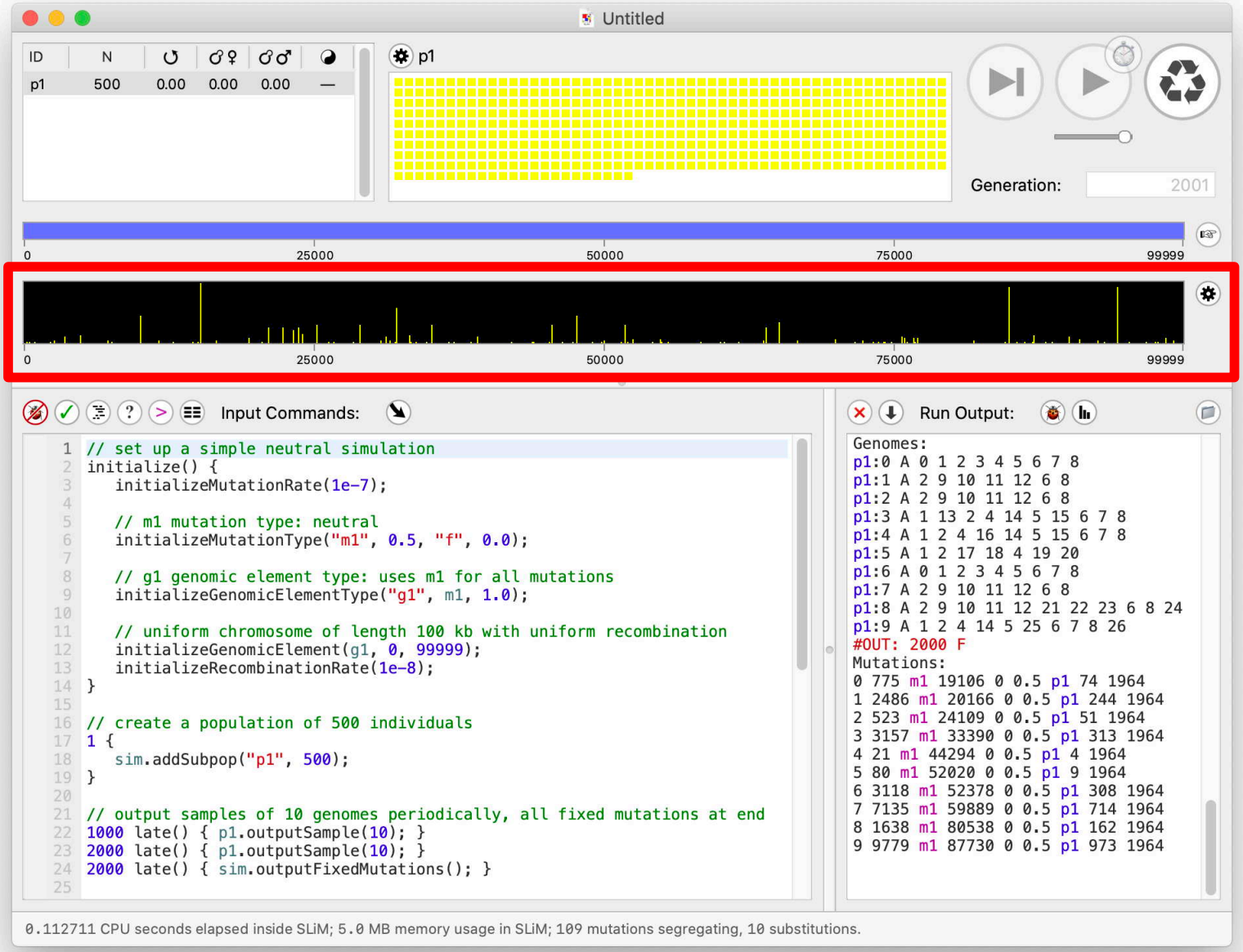
Modeling in SLiMgui



Modeling in SLiMgui



Modeling in SLiMgui



Modeling in SLiMgui

Untitled

ID	N	↺	♂♀	♂♂	◕
p1	500	0.00	0.00	0.00	—

p1

Generation: 2001

0 25000 50000 75000 99999

0 25000 50000 75000 99999

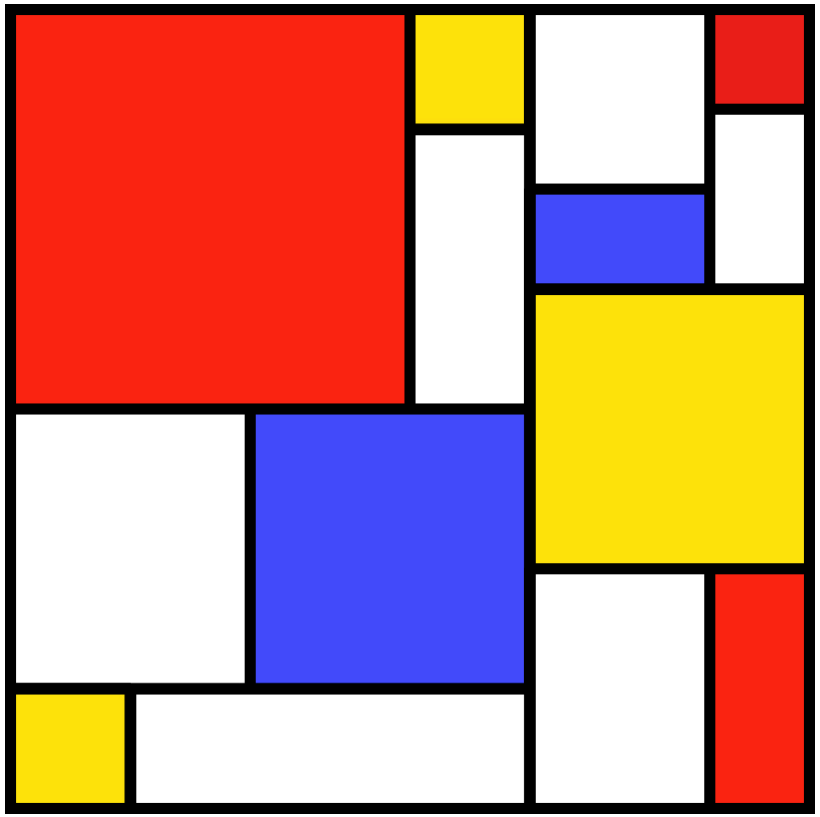
Input Commands:

```
// set up a simple neutral simulation
initialize() {
   initializeMutationRate(1e-7);

   // m1 mutation type: neutral
   initializeMutationType("m1", 0.5, "f", 0.0);

   // g1 genomic element type: uses m1 for all mutations
   initializeGenomicElementType("g1", m1, 1.0);

   // uniform chromosome of length 100 kb with uniform recombination
   initializeGenomicElement(g1, 0, 99999);
   initializeRecombinationRate(1e-8);
}

// create a population of 500 individuals
1 {
   sim.addSubpop("p1", 500);
}

// output samples of 10 genomes periodically, all fixed mutations at end
1000 late() { p1.outputSample(10); }
2000 late() { p1.outputSample(10); }
2000 late() { sim.outputFixedMutations(); }
```

Run Output:

```
Genomes:
p1:0 A 0 1 2 3 4 5 6 7 8
p1:1 A 2 9 10 11 12 6 8
p1:2 A 2 9 10 11 12 6 8
p1:3 A 1 13 2 4 14 5 15 6 7 8
p1:4 A 1 2 4 16 14 5 15 6 7 8
p1:5 A 1 2 17 18 4 19 20
p1:6 A 0 1 2 3 4 5 6 7 8
p1:7 A 2 9 10 11 12 6 8
p1:8 A 2 9 10 11 12 21 22 23 6 8 24
p1:9 A 1 2 4 14 5 25 6 7 8 26
#OUT: 2000 F
Mutations:
0 775 m1 19106 0 0.5 p1 74 1964
1 2486 m1 20166 0 0.5 p1 244 1964
2 523 m1 24109 0 0.5 p1 51 1964
3 3157 m1 33390 0 0.5 p1 313 1964
4 21 m1 44294 0 0.5 p1 4 1964
5 80 m1 52020 0 0.5 p1 9 1964
6 3118 m1 52378 0 0.5 p1 308 1964
7 7135 m1 59889 0 0.5 p1 714 1964
8 1638 m1 80538 0 0.5 p1 162 1964
9 9779 m1 87730 0 0.5 p1 973 1964
```

0.112711 CPU seconds elapsed inside SLiM; 5.0 MB memory usage in SLiM; 109 mutations segregating, 10 substitutions.



Copyright © 2020–2025 Benjamin C. Haller. All Rights Reserved.